管理和保护应用程序

Astra Control Service

NetApp
January 12, 2022

This PDF was generated from https://docs.netapp.com/zh-cn/astra-control-service/use/manage-apps.html on January 12, 2022. Always check docs.netapp.com for the latest.

目录

管理和保护应用程序 1
开始管理应用程序 1
通过快照和备份保护应用程序 9
还原应用程序 14
克隆和迁移应用程序 15
管理应用程序执行挂钩 16

管理和保护应用程序

开始管理应用程序

你先请 "将 Kubernetes 集群添加到 Astra Control"，您可以在集群上安装应用程序（在 Astra Control 之外），然后转到 Astra Control 中的应用程序页面开始管理这些应用程序。

应用程序管理要求

请考虑以下 Astra Control 应用程序管理要求：

- * 许可 *： 要使用 Astra 控制中心管理应用程序，您需要获得 Astra 控制中心许可证。
- * 命名空间 *： Astra Control 要求一个应用程序不能跨越多个命名空间，但一个命名空间可以包含多个应用程序。
- * 存储类 *： 如果您安装的应用程序明确设置了 StorageClass ，并且需要克隆该应用程序，则克隆操作的目标集群必须具有最初指定的 StorageClass 。将显式设置了 StorageClass 的应用程序克隆到不具有相同 StorageClass 的集群将失败。
- * Kubernetes Resources*： 使用非 Astra Control 收集的 Kubernetes 资源的应用程序可能没有完整的应用程序数据管理功能。Astra Control 收集以下 Kubernetes 资源：
  - ClusterRole
  - ClusterRoleBinding.
  - 配置映射
  - 自定义资源定义
  - 自定义资源
  - DemonSet
  - 部署
  - DeploymentConfig
  - 传入
  - MutatingWebhook
  - PersistentVolumeClaim
  - POD
  - ReplicaSet
  - RoleBinding.
  - Role
  - 路由
  - 机密
  - 服务
  - ServiceAccount
  - 状态集

- 验证 Webhook

支持的应用程序安装方法

您可以使用以下方法安装要使用 Astra Control 管理的应用程序：

- * 清单文件 * ： Astra Control 支持使用 kubectl 从清单文件安装的应用程序。例如：

```
kubectl apply -f myapp.yaml
```

- * Helm 3* ： 如果使用 Helm 安装应用程序，则 Astra Control 需要 Helm 版本 3 。完全支持管理和克隆随 Helm 3 安装的应用程序（或从 Helm 2 升级到 Helm 3 ）。不支持管理随 Helm 2 安装的应用程序。
- * 操作员部署的应用程序 * ： Astra Control 支持使用命名空间范围的运算符安装的应用程序。这些操作员通常采用 " 按价值传递 " 架构，而不是 " 按参考传递 " 架构。以下是一些遵循这些模式的操作员应用程序：
  - "Apache K8ssandra"
  - "Jenkins CI"
  - "Percona XtraDB 集群"

请注意， Astra Control 可能无法克隆使用 " 按参考传递 " 架构设计的运算符（例如 CockroachDB 运算符）。在这些类型的克隆操作期间，克隆的操作员会尝试引用源操作员提供的 Kubernetes 机密，尽管在克隆过程中他们拥有自己的新机密。克隆操作可能会失败，因为 Astra Control 不知道源运算符中的 Kubernetes 密钥。

> 操作员及其安装的应用程序必须使用相同的命名空间；您可能需要为操作员修改部署 .yaml 文件，以确保情况确实如此。

在集群上安装应用程序

现在，您已将集群添加到 Astra Control 中，您可以在集群上安装应用程序。默认情况下，将在新存储类上配置永久性卷。Pod 联机后，您可以使用 Astra Control 管理应用程序。

只有当存储位于由 Astra Control 安装的存储类上时， Astra Control 才会管理有状态应用程序。

- "了解 GKEE 集群的存储类"
- "了解 AKS 集群的存储类"

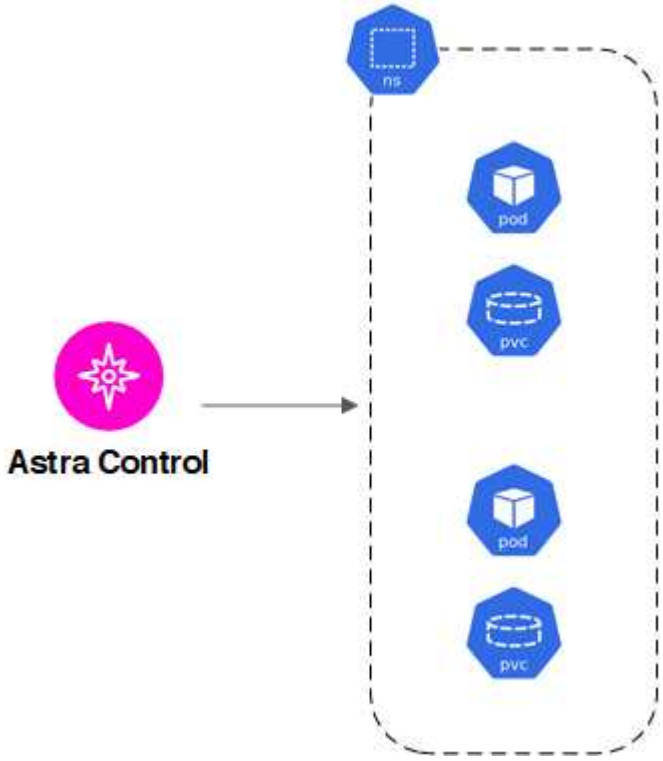
有关通过 Helm 图表部署常见应用程序的帮助，请参见以下内容：

- "从 Helm 图表部署 MariaDB"
- "从 Helm 图表部署 MySQL"
- "从 Helm 图表部署 Postgres"
- "从 Helm 图表中部署 Jenkins"

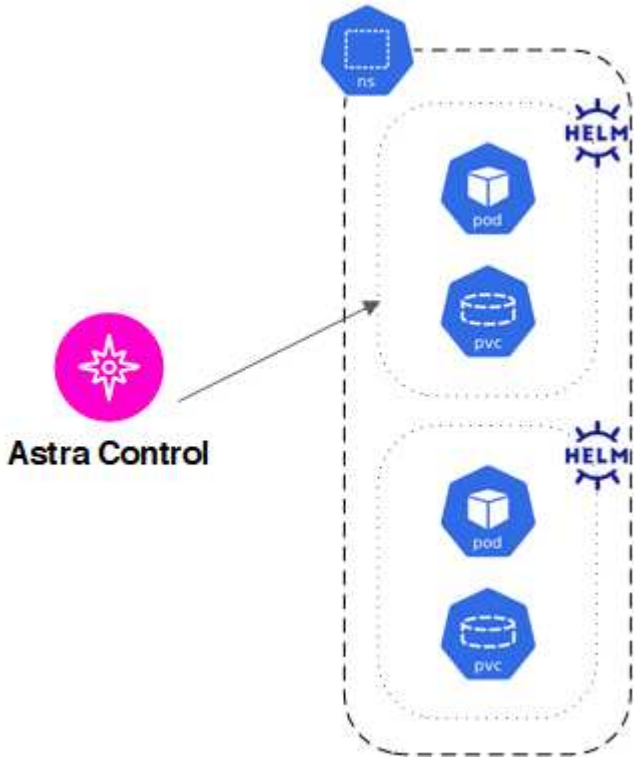
管理应用程序

当 Astra Control 发现集群上运行的应用程序时，这些应用程序将不受管理，直到您选择要如何管理它们为止。Astra Control 中的受管应用程序可以是以下任一项：

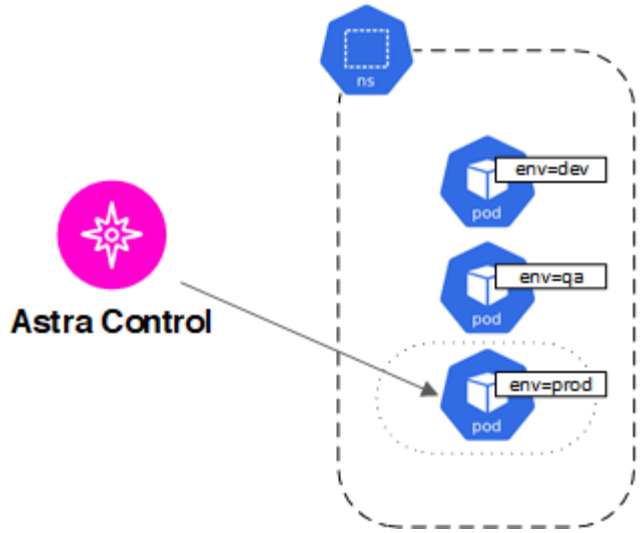
- 命名空间，包括该命名空间中的所有资源



- 在命名空间中使用 helm3 部署的单个应用程序



- 一组通过 Kubernetes 标签标识的资源（在 Astra Control 中称为 *custom app* ）



以下各节介绍如何使用这些选项管理应用程序。

按命名空间管理应用程序

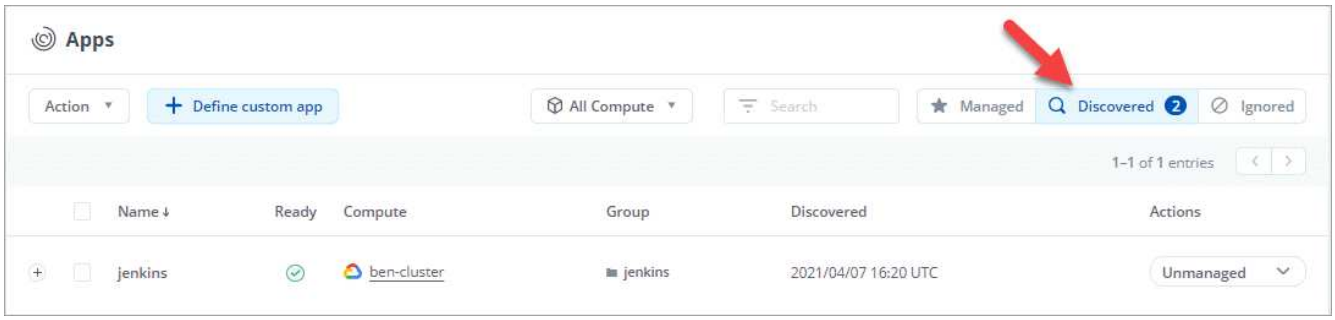
" 应用程序 " 页面的 * 已发现 * 部分显示命名空间以及这些命名空间中 Helm 安装的应用程序或自定义标记的应用程序。您可以选择单独管理每个应用程序，也可以选择在命名空间级别管理每个应用程序。这一切都可以细化到数据保护操作所需的粒度级别。

例如，您可能希望为 "Maria" 设置一个每周节奏的备份策略，但您可能需要比该策略更频繁地备份 "MariaDB" （位于同一命名空间中）。根据这些需求，您需要单独管理应用程序，而不是在一个命名空间下进行管理。

虽然 Astra Control 允许您单独管理层次结构的两个级别（命名空间和该命名空间中的应用程序），但最佳做法是选择一个或另一个。如果在命名空间和应用程序级别同时执行操作，则在 Astra Control 中执行的操作可能会失败。

步骤

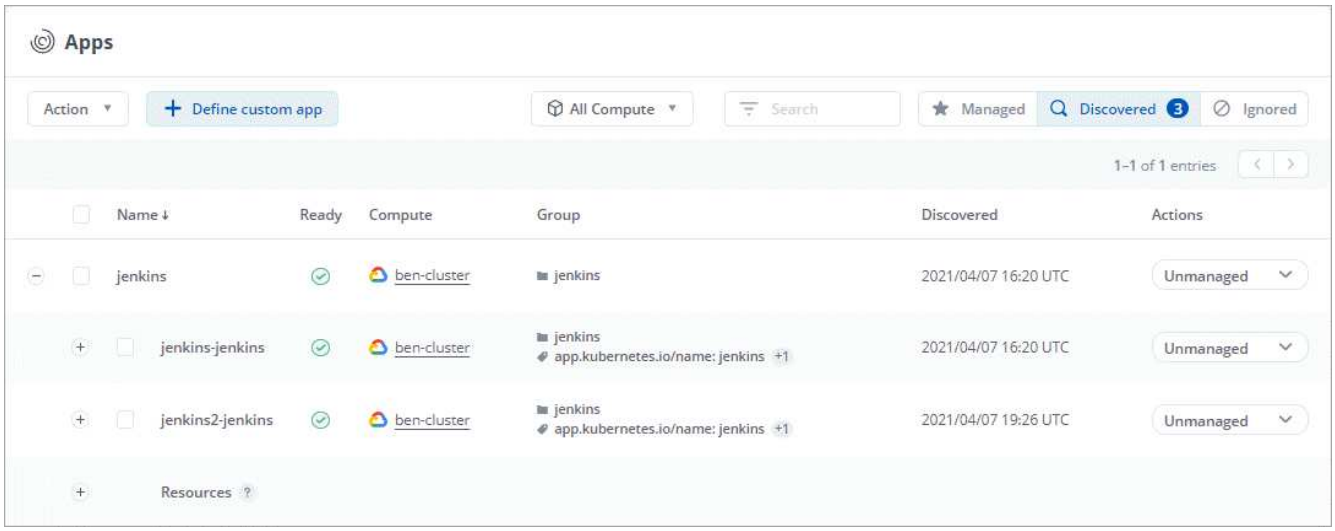
1. 选择 * 应用程序 * ，然后选择 * 已发现 * 。



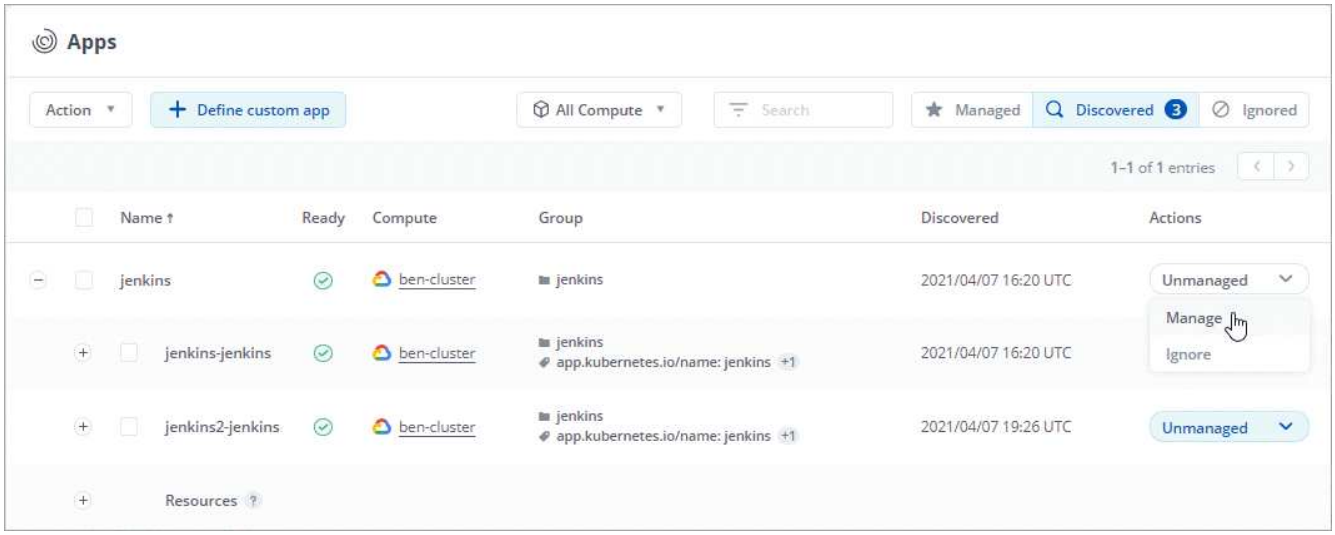
2. 查看已发现的命名空间列表并展开命名空间以查看应用程序和关联资源。

   Astra Control 会在命名空间中显示 Helm 应用程序和自定义标记的应用程序。如果 Helm 标签可用，则会使用标记图标来指定这些标签。

   以下是一个命名空间中包含两个应用程序的示例：

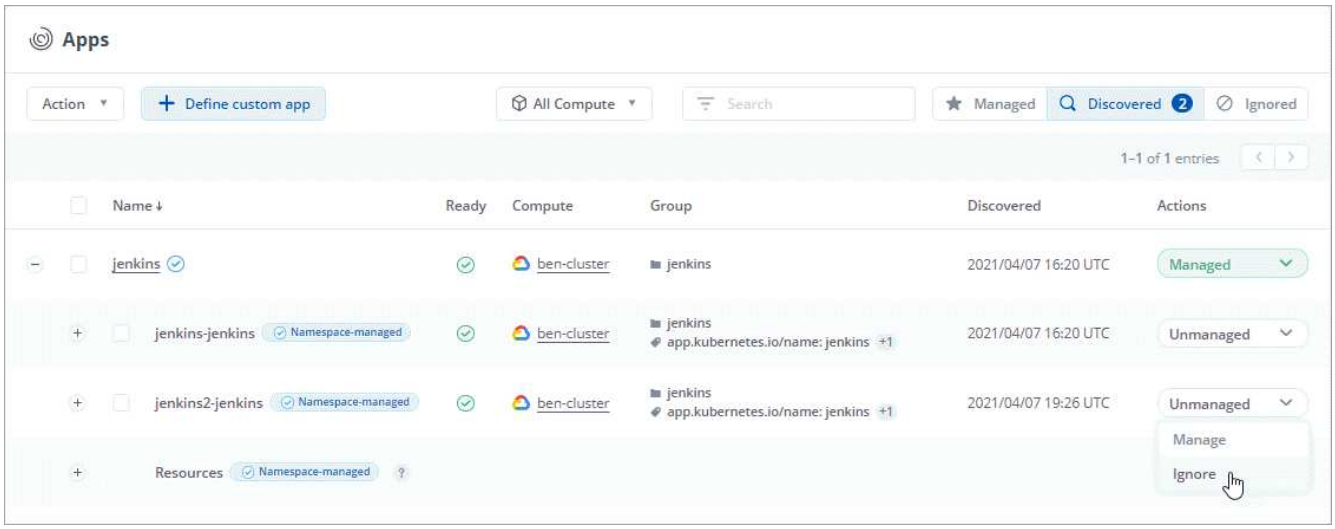


3. 确定是单独管理每个应用程序，还是在命名空间级别管理每个应用程序。
4. 在层次结构中的所需级别，选择 * 操作 * 列中的下拉列表，然后选择 * 管理 * 。



5. 如果您不想管理某个应用程序，请选择所需应用程序的 * 操作 * 列中的下拉列表，然后选择 * 忽略 * 。

   例如，如果您希望同时管理 "Jenkins " 命名空间下的所有应用程序，以便它们具有相同的快照和备份策略，则可以管理此命名空间并忽略此命名空间中的应用程序：



您选择管理的应用程序现在可从 * 受管 * 选项卡访问。任何被忽略的应用程序都将移至 * 已忽略 * 选项卡。理想情况下， " 已发现 " 选项卡将显示零个应用程序，以便在安装新应用程序后更容易找到和管理这些应用程序。

按 **Kubernetes** 标签管理应用程序

Astra Control 在应用程序页面顶部包含一个名为 * 定义自定义应用程序 * 的操作。您可以使用此操作管理使用 Kubernetes 标签标识的应用程序。 "了解有关通过 Kubernetes 标签定义应用程序的更多信息"。

步骤

1. 选择 * 应用程序 > 定义自定义应用程序 * 。
2. 在 * 定义自定义应用程序 * 对话框中，提供管理该应用程序所需的信息：

    a. * 新建应用程序 * ： 输入应用程序的显示名称。

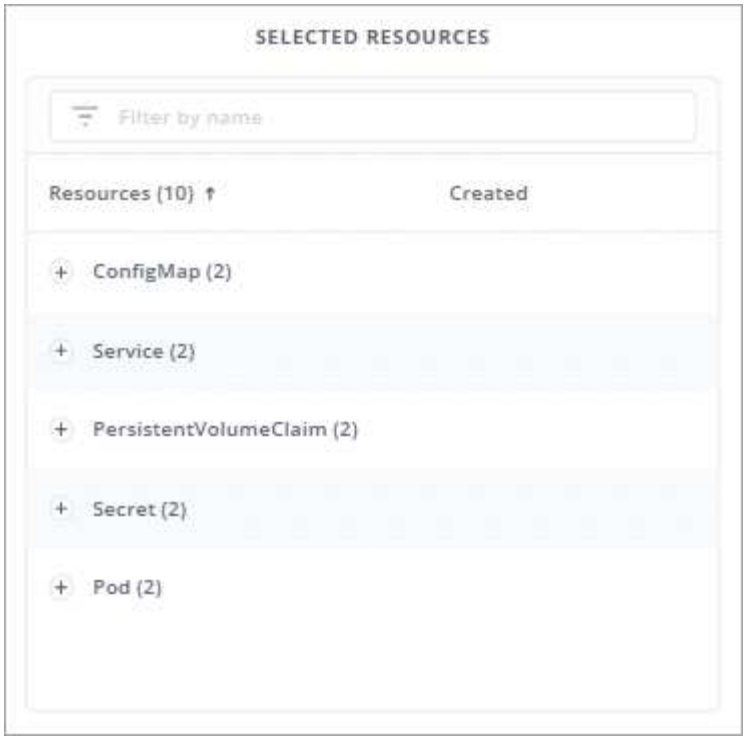
    b. * 集群 * ： 选择应用程序所在的集群。

    c. * 命名空间： * 选择应用程序的命名空间。

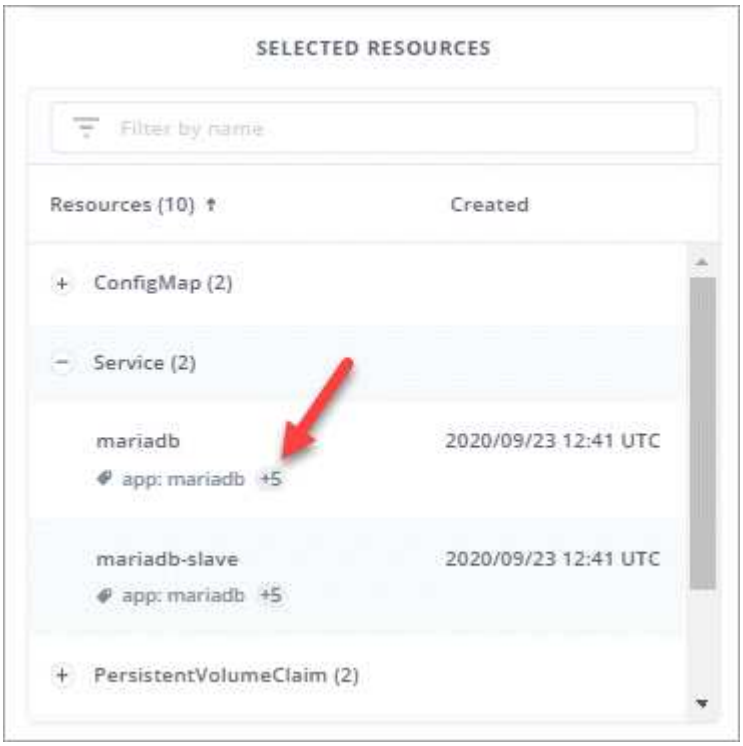
    d. * 标签： * 输入标签或从以下资源中选择标签。

    e. * 选定资源 * ： 查看和管理要保护的选定 Kubernetes 资源（ Pod ，机密，永久性卷等）。

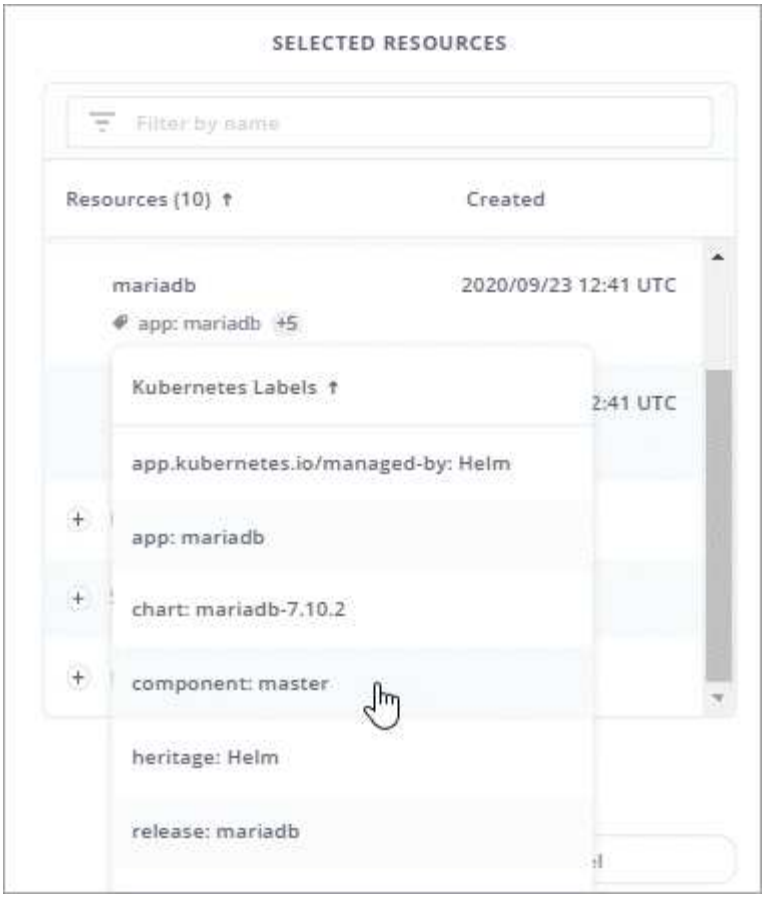
    以下是一个示例：



- 通过展开资源并选择标签数量来查看可用标签。

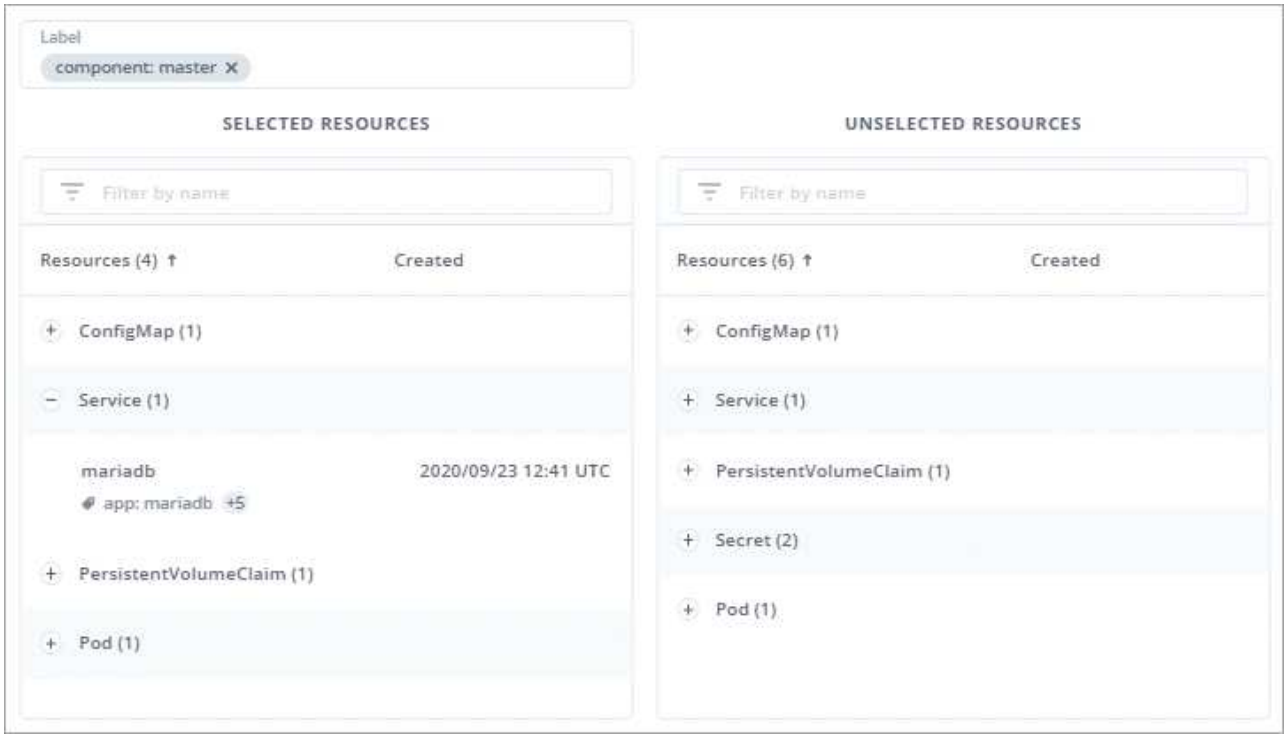


- 选择一个标签。



选择标签后，它将显示在 * 标签 * 字段中。Astra Control 还会更新 * 未选定资源 * 部分，以显示与选定标签不匹配的资源。

a. * 未选择资源 *： 验证您不想保护的应用程序资源。

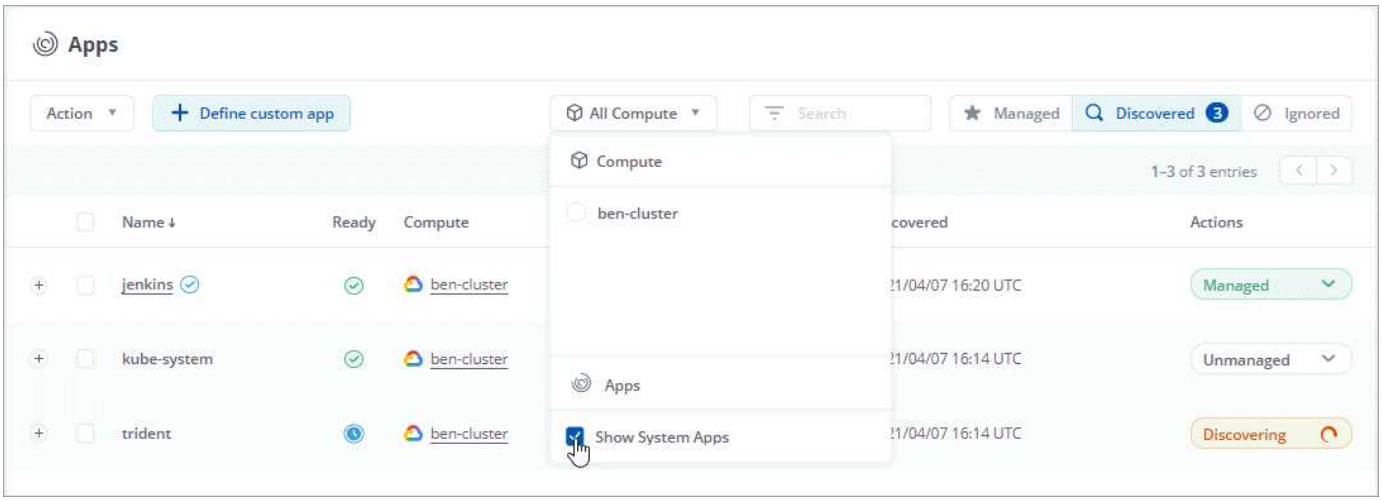


3. 选择 * 定义自定义应用程序 * 。

使用 Astra Control 可以管理应用程序。现在，您可以在 * 受管 * 选项卡中找到它。

系统应用程序如何？

Astra Control 还会发现 Kubernetes 集群上运行的系统应用程序。您可以通过筛选应用程序列表来查看这些应用程序。



默认情况下，我们不会向您显示这些系统应用程序，因为您很少需要备份这些应用程序。

通过快照和备份保护应用程序

通过使用自动保护策略或临时创建快照和备份来保护应用程序。

快照和备份

snapshot 是应用程序的时间点副本，它与应用程序存储在同一个已配置卷上。通常速度较快。本地快照用于将应用程序还原到较早的时间点。

backup 存储在云中的对象存储上。与本地快照相比，备份速度可能较慢。但是，可以在云中的各个区域访问这些应用程序，以实现应用程序迁移。您还可以选择较长的备份保留期限。

> *You can't be Fully protected until you have a recent backup* 。这一点非常重要，因为备份存储在对象存储中，而不是永久性卷。如果发生故障或意外事件会擦除集群及其永久性存储，则需要备份才能恢复。快照无法让您恢复。

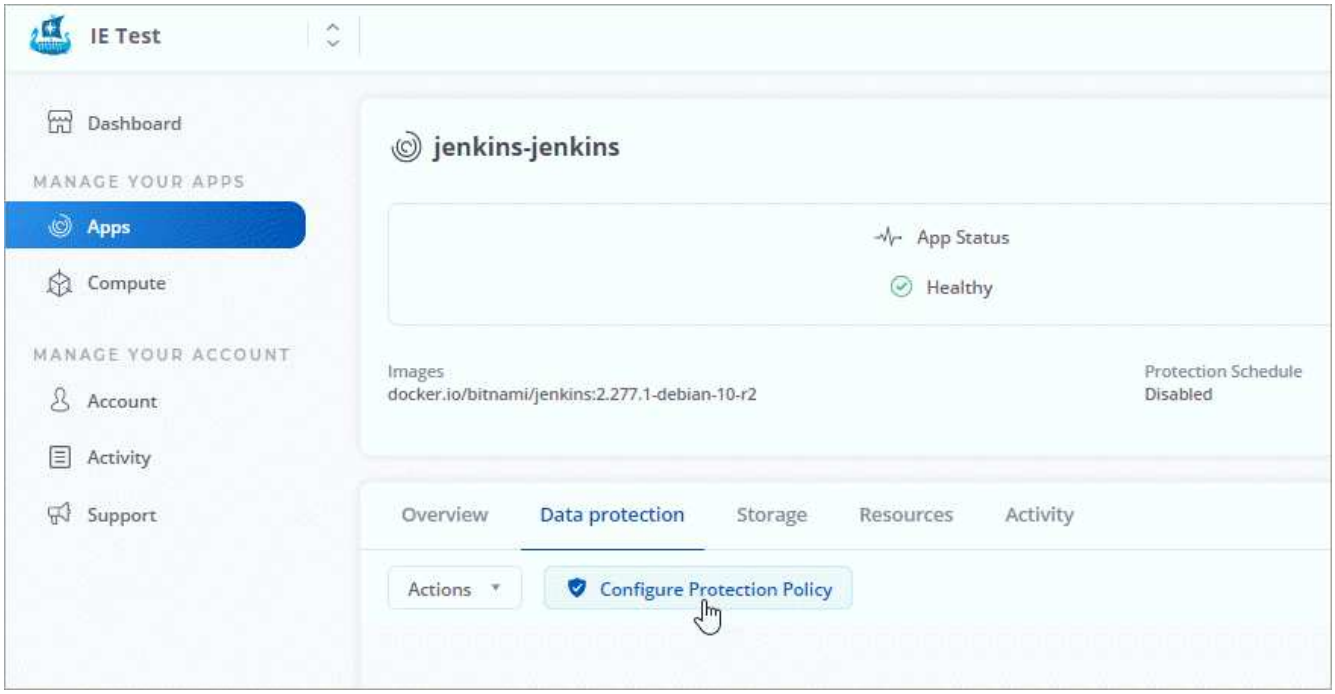
配置保护策略

保护策略通过按定义的计划创建快照，备份或这两者来保护应用程序。您可以选择每小时，每天，每周和每月创建快照和备份，并且可以指定要保留的副本数。

步骤

1. 选择 * 应用程序 * ，然后选择受管应用程序的名称。

2. 选择 * 数据保护 * 。
3. 选择 * 配置保护策略 * 。

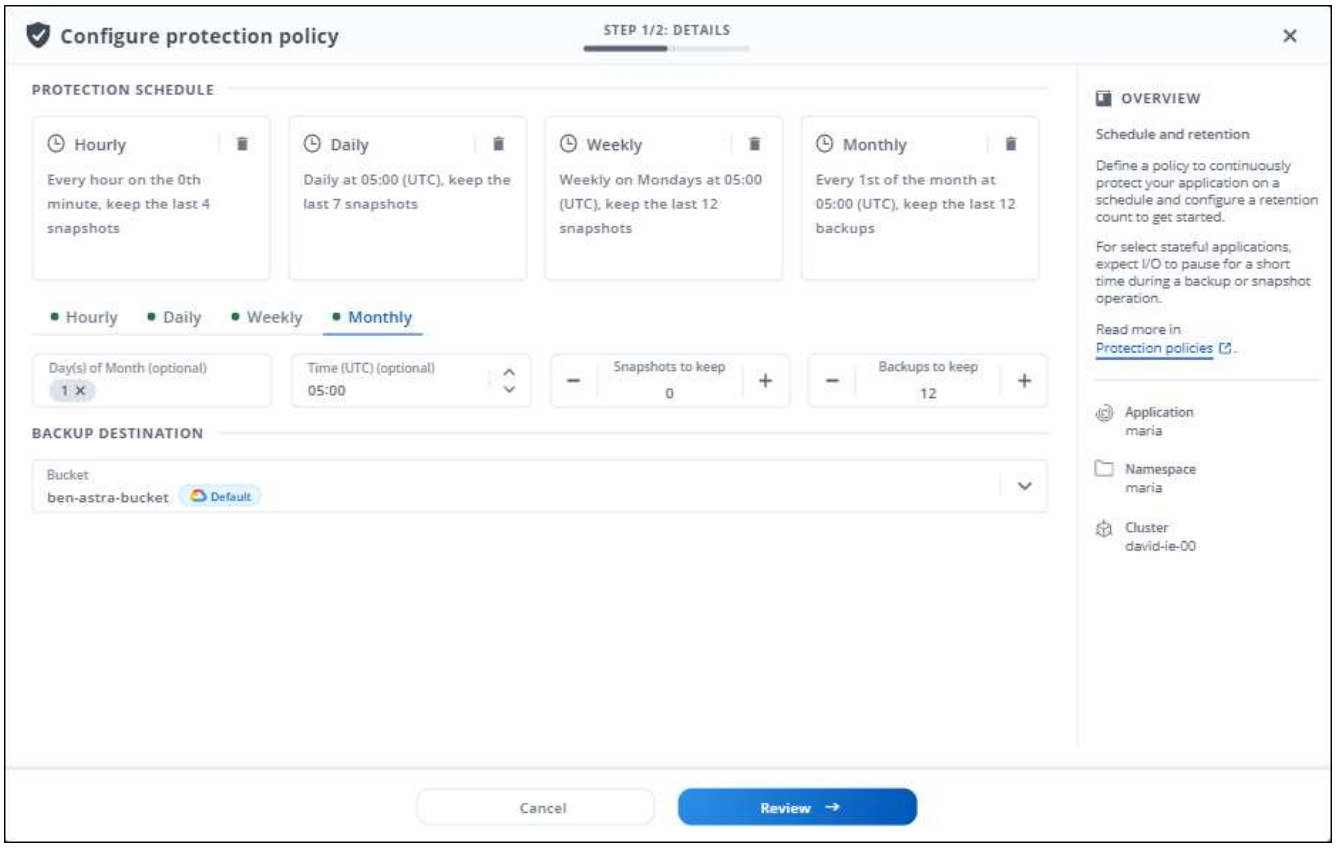


4. 通过选择每小时，每天，每周和每月计划要保留的快照和备份数量来定义保护计划。

   您可以同时定义每小时，每天，每周和每月计划。在为快照和备份设置保留级别之前，计划不会变为活动状态。

   在为备份设置保留级别时，您可以选择要将备份存储到的存储分段。

   以下示例将为快照和备份设置四个保护计划：每小时，每天，每周和每月。



5. 选择 * 审阅 * 。
6. 选择 * 配置 * 。

   下面是一个视频，其中介绍了每个步骤。

   ► https://docs.netapp.com/zh-cn/astra-control-service/media/use/video-set-protection-policy.mp4 *(video)*

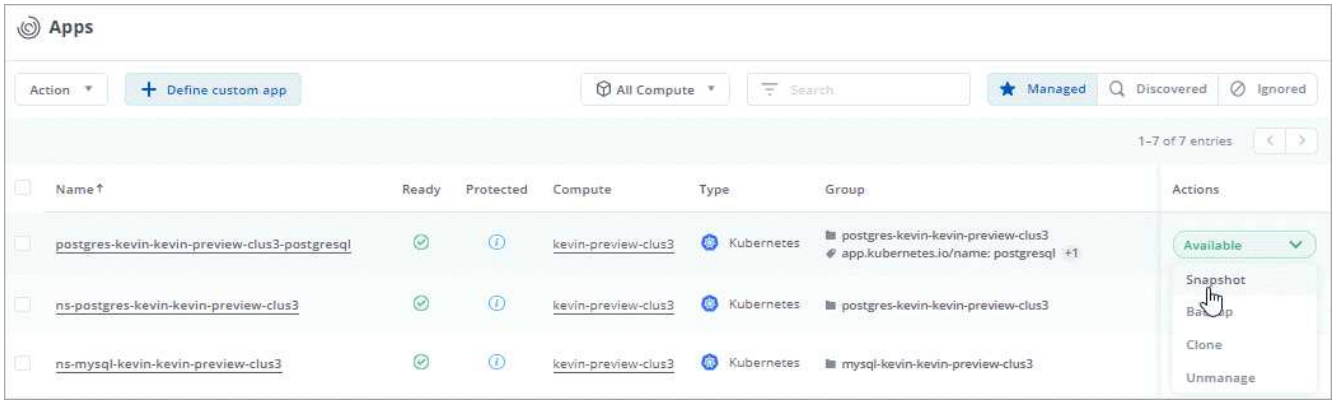
Astra Control 通过使用您定义的计划和保留策略创建和保留快照和备份来实施数据保护策略。

创建快照

您可以随时创建按需快照。

步骤

1. 选择 * 应用程序 * 。
2. 在 * 操作 * 列中为所需应用程序选择下拉列表。
3. 选择 * 快照 * 。



4. 自定义快照的名称，然后选择 * 查看信息 * 。
5. 查看快照摘要并选择 * 快照应用程序 * 。

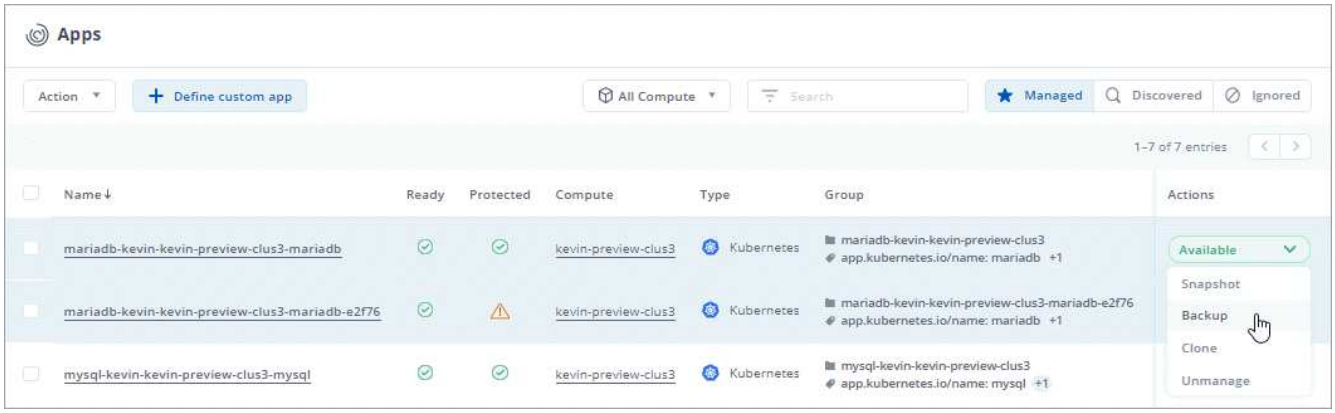
Astra Control 创建应用程序的快照。

创建备份

您也可以随时备份应用程序。

步骤

1. 选择 * 应用程序 * 。
2. 在 * 操作 * 列中为所需应用程序选择下拉列表。
3. 选择 * 备份 * 。



4. 自定义备份的名称，选择是否从现有快照备份应用程序，然后选择 * 查看信息 * 。
5. 查看备份摘要并选择 * 备份应用程序 * 。

Astra Control 会创建应用程序的备份。

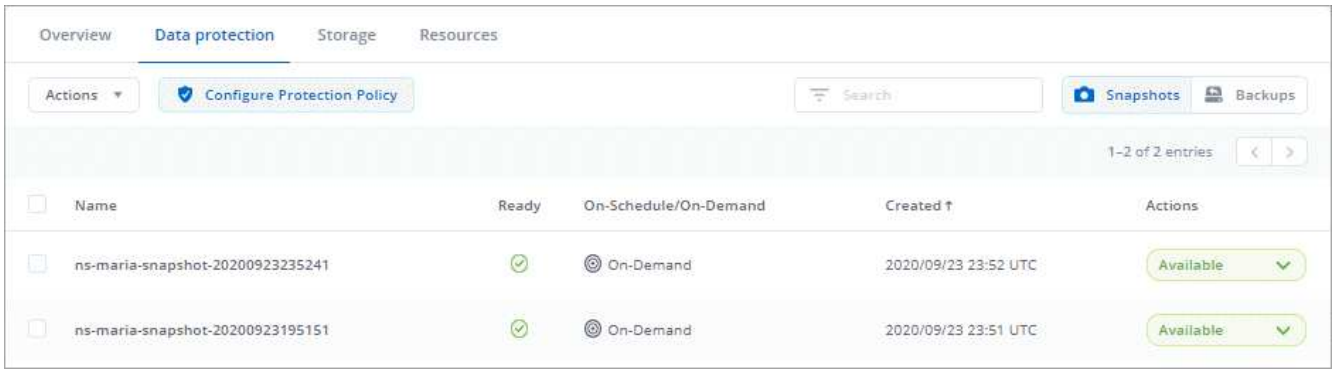
查看快照和备份

您可以从数据保护选项卡查看应用程序的快照和备份。

步骤

1. 选择 * 应用程序 * ，然后选择受管应用程序的名称。
2. 选择 * 数据保护 * 。

   默认情况下会显示快照。



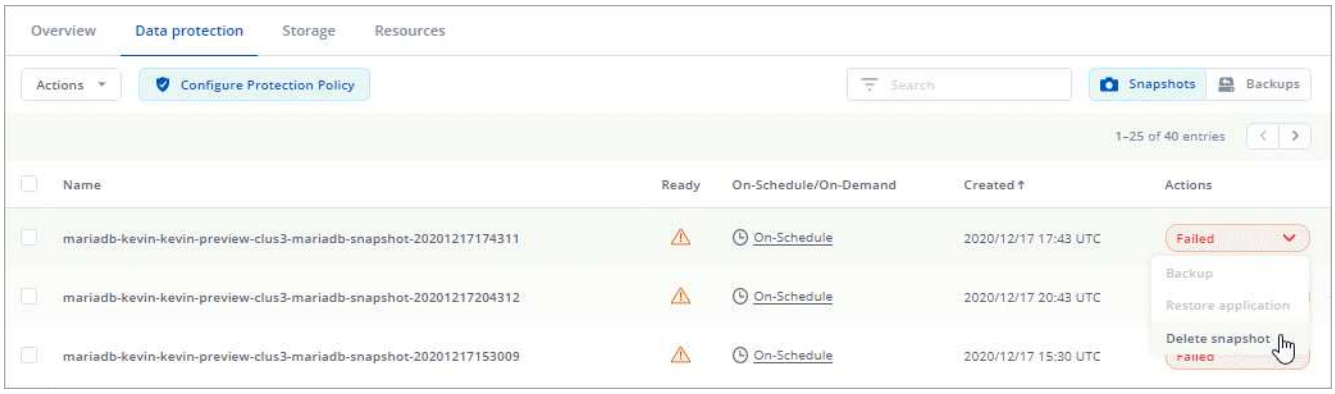
3. 选择 * 备份 * 可查看备份列表。

删除快照

删除不再需要的计划快照或按需快照。

步骤

1. 选择 * 应用程序 * ，然后选择受管应用程序的名称。
2. 选择 * 数据保护 * 。
3. 在 * 操作 * 列中选择所需快照的下拉列表。
4. 选择 * 删除快照 * 。



5. 键入快照的名称以确认删除，然后选择 * 是，删除快照 * 。

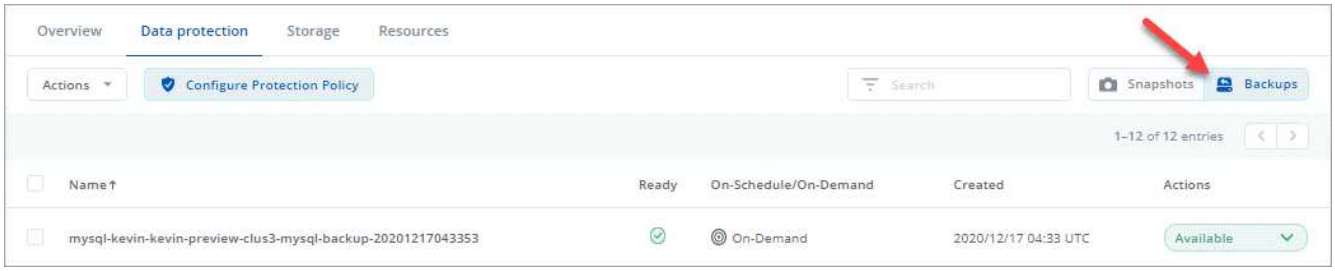
Astra Control 会删除快照。

删除备份

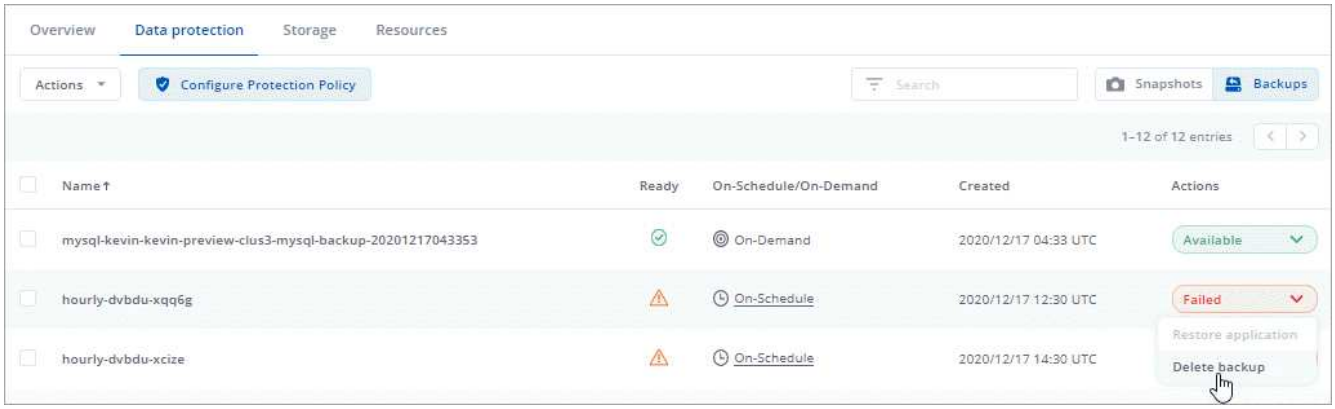
删除不再需要的计划备份或按需备份。

1. 选择 * 应用程序 * ，然后选择受管应用程序的名称。

2. 选择 * 数据保护 * 。
3. 选择 * 备份 * 。



4. 在 * 操作 * 列中选择所需备份的下拉列表。
5. 选择 * 删除备份 * 。



6. 键入备份的名称以确认删除，然后选择 * 是，删除备份 * 。

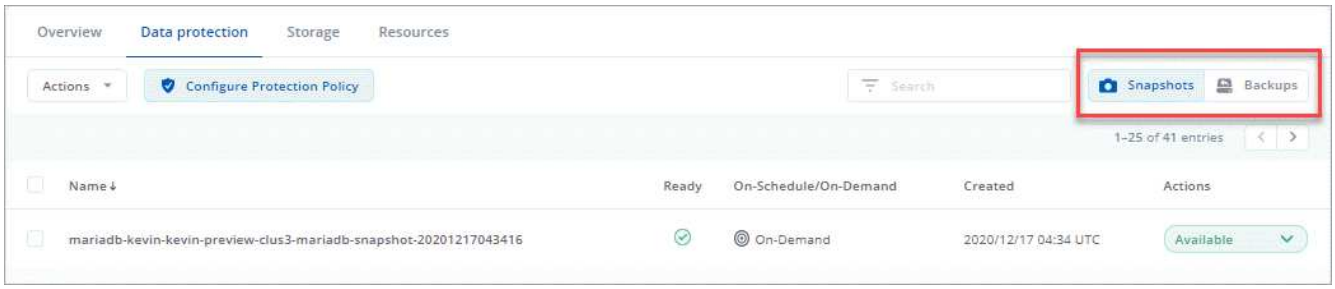
Astra Control 会删除备份。

还原应用程序

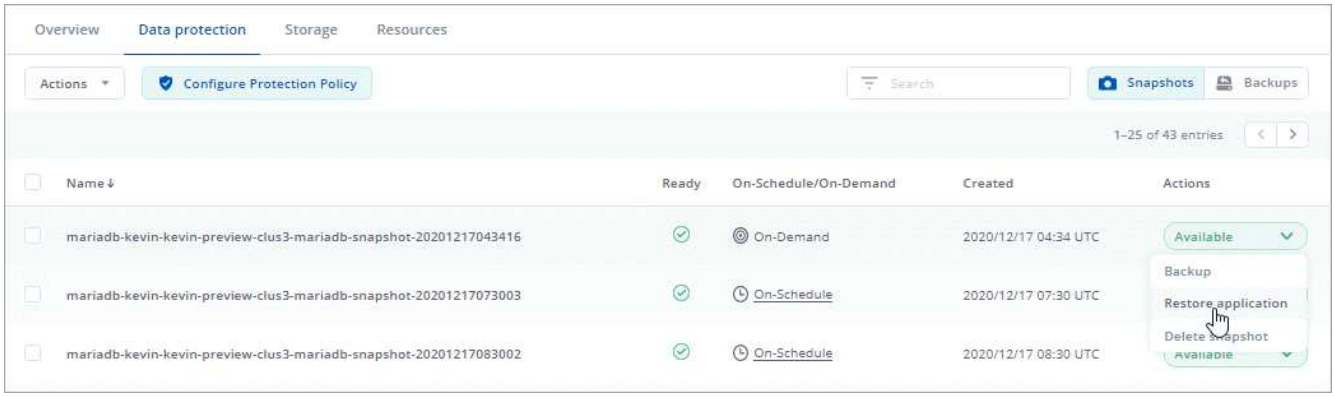
Astra Control 可以从快照或备份还原应用程序配置和永久性存储。永久性存储备份会从对象存储传输，因此从现有备份还原的速度最快。

步骤

1. 选择 * 应用程序 * ，然后选择受管应用程序的名称。
2. 选择 * 数据保护 * 。
3. 如果要从快照还原，请保持选中 * 快照 * 。否则，请选择 * 备份 * 以从备份中还原。



4. 在 * 操作 * 列中选择要从中还原的快照或备份的下拉列表。
5. 选择 * 还原应用程序 * 。



6. * 还原详细信息 * ：指定已还原应用程序的详细信息。要原位还原应用程序（将应用程序还原到其自身的早期版本），请选择当前运行的同一命名空间和目标集群：
   - 输入应用程序的命名空间。
   - 选择应用程序的目标集群。
   - 单击 * 审阅 * 。
7. * 还原摘要 * ：查看有关还原操作的详细信息，键入 "restore" ，然后选择 * 还原 * 。

Astra Control 会根据您提供的信息还原应用程序。如果已原位还原应用程序，则会删除并重新创建关联的永久性卷。

克隆和迁移应用程序

克隆现有应用程序以在同一个 Kubernetes 集群或另一个集群上创建重复的应用程序。如果您需要将应用程序和存储从一个 Kubernetes 集群移动到另一个集群，则克隆可以助您一臂之力。例如，您可能希望通过 CI/CD 管道以及在 Kubernetes 命名空间之间移动工作负载。

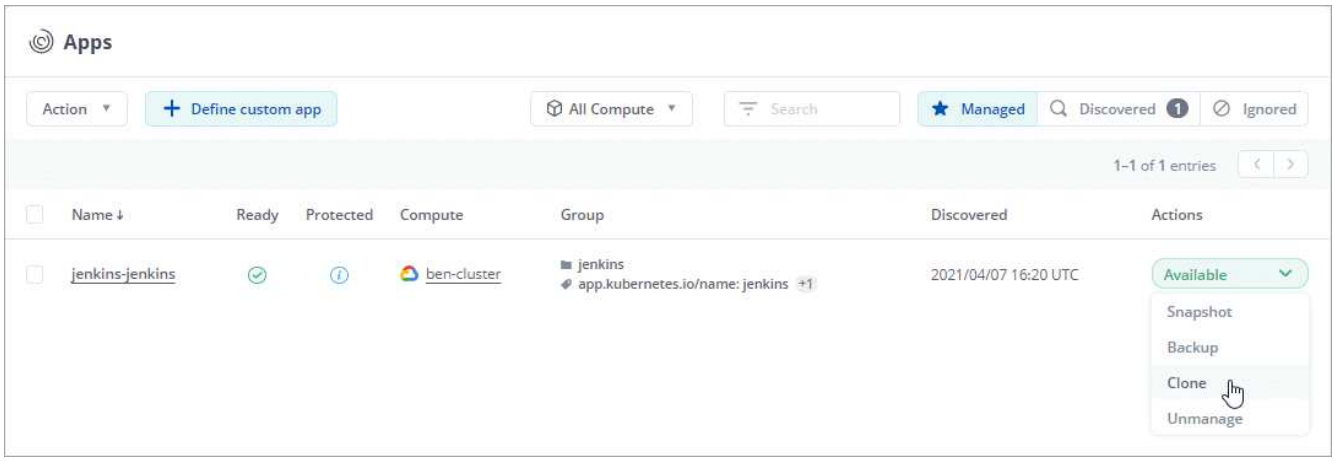
当 Astra Control 克隆应用程序时，它会为您的应用程序配置和永久性存储创建一个克隆。

> 如果克隆操作员部署的 Jenkins CI 实例，则需要手动还原永久性数据。这是应用程序部署模式的一个限制。

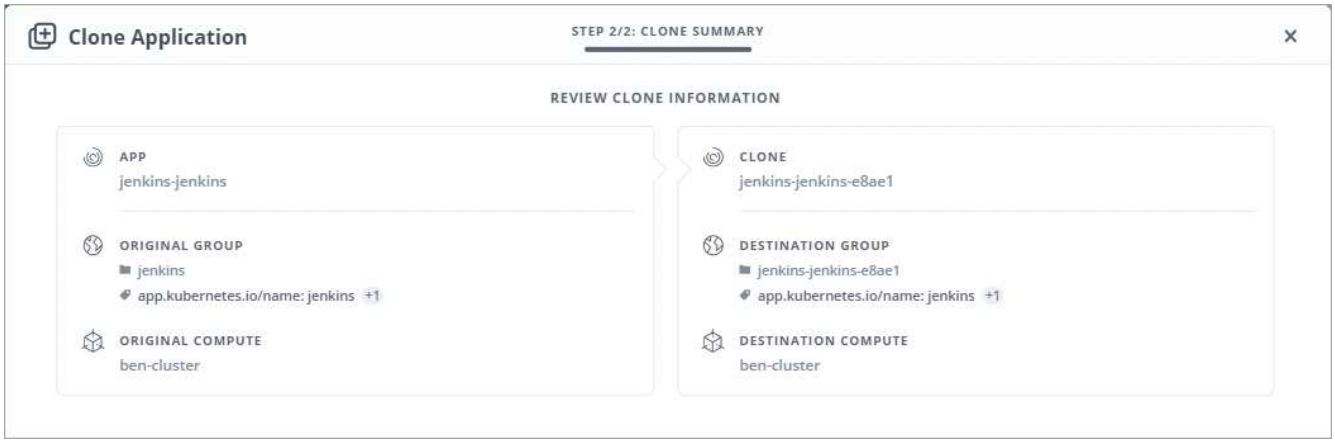
> 如果您部署的应用程序明确设置了 StorageClass ，并且需要克隆该应用程序，则目标集群必须具有最初指定的 StorageClass 。将显式设置了 StorageClass 的应用程序克隆到不具有相同 StorageClass 的集群将失败。

步骤

1. 选择 * 应用程序 * 。
2. 在 * 操作 * 列中为所需应用程序选择下拉列表。
3. 选择 * 克隆 * 。



4. * 克隆详细信息 * ：指定克隆的详细信息：
   - 保留默认名称和命名空间，或者对其进行编辑。
   - 选择克隆的目标集群。
   - 选择是要从现有快照还是备份创建克隆。如果不选择此选项，则 Astra Control 将根据应用程序的当前状态创建克隆。
5. * 克隆摘要 * ：查看有关克隆的详细信息，然后选择 * 克隆应用程序 * 。



Astra Control 会根据您提供的信息克隆该应用程序。

管理应用程序执行挂钩

执行钩是一种自定义脚本，您可以在托管应用程序快照之前或之后运行该脚本。例如，如果您有一个数据库应用程序，则可以使用执行挂钩在快照之前暂停所有数据库事务，并在快照完成后恢复事务。这样可以确保应用程序一致的快照。

默认执行挂钩和正则表达式

对于某些应用程序， Astra Control 附带了 NetApp 提供的默认执行挂钩，用于处理快照前后的冻结和解冻操作。Astra Control 使用正则表达式将应用程序的容器映像与以下应用程序匹配：

- MariaDB

    - 匹配正则表达式： \bmariadb\b
- MySQL
    - 匹配正则表达式： \bmysql\b
- PostgreSQL
    - 匹配正则表达式： \bpostgresql\b

如果存在匹配项，则 NetApp 为该应用程序提供的默认执行挂钩将显示在该应用程序的活动执行挂钩列表中，这些挂钩将在该应用程序创建快照时自动运行。如果某个自定义应用程序的映像名称类似，恰好与其中一个正则表达式匹配（并且您不想使用默认执行挂钩），则可以更改映像名称， 或者禁用该应用程序的默认执行连接，而改用自定义连接。

您不能删除或修改默认执行挂钩。

有关自定义执行挂钩的重要注意事项

在为应用程序规划执行挂钩时，请考虑以下几点。

- Astra Control 要求以可执行 Shell 脚本的格式编写执行挂钩。
- 脚本大小限制为 128 KB 。
- Astra Control 使用执行挂钩设置和任何匹配条件来确定哪些挂钩适用于快照。
- 所有执行挂机故障均为软故障；即使某个挂机发生故障，仍会尝试使用其他挂机和快照。但是，如果挂机发生故障，则会在 * 活动 * 页面事件日志中记录一个警告事件。
- 要创建，编辑或删除执行挂钩，您必须是具有所有者，管理员或成员权限的用户。
- 如果执行挂机运行时间超过 25 分钟，则此挂机将失败，从而创建返回代码为不适用的事件日志条目。任何受影响的快照都将超时并标记为失败，并会生成一个事件日志条目，用于记录超时情况。

> 由于执行挂钩通常会减少或完全禁用其所运行的应用程序的功能，因此您应始终尽量缩短自定义执行挂钩运行所需的时间。

运行快照时，执行钩事件按以下顺序发生：

1. NetApp 提供的任何适用的默认快照前执行挂钩都会在相应的容器上运行。
2. 任何适用的自定义快照前执行挂钩都会在相应的容器上运行。您可以根据需要创建和运行任意数量的自定义预快照挂钩，但在创建快照之前执行这些挂钩的顺序既不能保证也不可配置。
3. 执行快照。
4. 任何适用的自定义快照后执行挂钩都会在相应的容器上运行。您可以根据需要创建和运行任意数量的自定义快照后挂钩，但这些挂钩在快照后的执行顺序既不能保证也不可配置。
5. NetApp 提供的任何适用的默认快照后执行挂钩都会在相应的容器上运行。

> 在生产环境中启用执行钩脚本之前，应始终对其进行测试。您可以使用 "kubectl exec" 命令方便地测试脚本。在生产环境中启用执行挂钩后，测试生成的快照以确保其一致。为此，您可以将应用程序克隆到临时命名空间，还原快照，然后测试应用程序。

查看现有执行挂钩

您可以查看应用程序的现有自定义或 NetApp 提供的默认执行挂钩。

步骤

1. 转到 * 应用程序 * ，然后选择受管应用程序的名称。
2. 选择 * 执行挂钩 * 选项卡。

   您可以在显示的列表中查看所有已启用或已禁用的执行挂钩。您可以查看挂机的状态，源以及运行时间（快照前或快照后）。要查看与执行挂钩相关的事件日志，请转到左侧导航区域中的 * 活动 * 页面。

创建自定义执行挂钩

您可以为应用程序创建自定义执行挂钩。请参见 "执行钩示例" 有关挂机示例。要创建执行挂钩，您需要拥有所有者，管理员或成员权限。

> 创建用作执行挂钩的自定义 Shell 脚本时，请务必在文件开头指定适当的 shell ，除非您正在运行 Linux 命令或提供可执行文件的完整路径。

步骤

1. 选择 * 应用程序 * ，然后选择受管应用程序的名称。
2. 选择 * 执行挂钩 * 选项卡。
3. 选择 * 添加新挂钩 * 。
4. 在 * 挂机详细信息 * 区域中，根据挂机应运行的时间，选择 * 预 Snapshot * 或 * 后 Snapshot * 。
5. 输入此挂钩的唯一名称。
6. （可选）输入执行期间传递到挂机的任何参数，在输入的每个参数之后按 Enter 键以记录每个参数。
7. 在 * 容器映像 * 区域中，如果此挂钩应针对应用程序中包含的所有容器映像运行，请启用 * 应用于所有容器映像 * 复选框。如果该挂钩只能作用于一个或多个指定的容器映像，请在 * 要匹配的容器映像名称 * 字段中输入容器映像名称。
8. 在 * 脚本 * 区域中，执行以下操作之一：
   - 上传自定义脚本。
     i. 选择 * 上传文件 * 选项。
     ii. 浏览到文件并上传。
     iii. 为脚本指定一个唯一名称。
     iv. （可选）输入其他管理员应了解的有关该脚本的任何注释。
   - 从剪贴板粘贴到自定义脚本中。
     i. 选择 * 从剪贴板粘贴 * 选项。
     ii. 选择文本字段并将脚本文本粘贴到字段中。
     iii. 为脚本指定一个唯一名称。
     iv. （可选）输入其他管理员应了解的有关该脚本的任何注释。

9. 选择 * 添加挂钩 * 。

禁用执行挂钩

如果要暂时阻止执行挂钩在应用程序快照之前或之后运行，可以禁用执行挂钩。要禁用执行挂钩，您需要拥有所有者，管理员或成员权限。

步骤

1. 选择 * 应用程序 * ，然后选择受管应用程序的名称。
2. 选择 * 执行挂钩 * 选项卡。
3. 为要禁用的挂钩选择 * 操作 * 下拉列表。
4. 选择 * 禁用 * 。

删除执行挂钩

如果您不再需要执行挂钩，则可以将其完全移除。要删除执行挂钩，您需要拥有所有者，管理员或成员权限。

步骤

1. 选择 * 应用程序 * ，然后选择受管应用程序的名称。
2. 选择 * 执行挂钩 * 选项卡。
3. 为要删除的挂钩选择 * 操作 * 下拉列表。
4. 选择 * 删除 * 。

执行钩示例

使用以下示例了解如何构建执行挂钩。您可以将这些挂钩用作模板或测试脚本。

简单的成功示例

这是一个简单的钩子示例，它成功地将消息写入标准输出和标准错误。

```
#!/bin/sh

# success_sample.sh
#
# A simple noop success hook script for testing purposes.
#
# args: None
#



#
# Writes the given message to standard output
#
# $* - The message to write
```

```
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample.sh"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

简单成功示例（ **bash** 版本）

这是一个简单的钩子示例，该钩子成功地将消息写入标准输出和标准错误，并写入 bash 。

```
#!/bin/bash

# success_sample.bash
#
# A simple noop success hook script for testing purposes.
#
```

```
# args: None


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample.bash"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

简单成功示例（**zsh** 版本）

这是一个简单的钩子示例，该钩子成功地将消息写入标准输出和标准错误，并写入 Z shell 。

```
#!/bin/zsh

# success_sample.zsh
#
# A simple noop success hook script for testing purposes.
#
# args: None
#


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}

#
# main
#

# log something to stdout
info "running success_sample.zsh"

# exit with 0 to indicate success
```

```
info "exit 0"
exit 0
```

成功使用参数示例

以下示例演示了如何在挂机中使用 args 。

```
#!/bin/sh

# success_sample_args.sh
#
# A simple success hook script with args for testing purposes.
#
# args: Up to two optional args that are echoed to stdout

#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
```

```
#

# log something to stdout
info "running success_sample_args.sh"


# collect args
arg1=$1
arg2=$2

# output args and arg count to stdout
info "number of args: $#"
info "arg1 ${arg1}"
info "arg2 ${arg2}"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

快照前 / 快照后挂钩示例

以下示例演示了如何对快照前和快照后挂钩使用同一脚本。

```
#!/bin/sh

# success_sample_pre_post.sh
#
# A simple success hook script example with an arg for testing purposes
# to demonstrate how the same script can be used for both a prehook and
posthook
#
# args: [pre|post]

# unique error codes for every error case
ebase=100
eusage=$((ebase+1))
ebadstage=$((ebase+2))
epre=$((ebase+3))
epost=$((ebase+4))


#
# Writes the given message to standard output
#
# $* - The message to write
#
```

```
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# Would run prehook steps here
#
prehook() {
    info "Running noop prehook"
    return 0
}

#
# Would run posthook steps here
#
posthook() {
    info "Running noop posthook"
    return 0
}


#
# main
#

# check arg
```

```
stage=$1
if [ -z "${stage}" ]; then
    echo "Usage: $0 <pre|post>"
    exit ${eusage}
fi

if [ "${stage}" != "pre" ] && [ "${stage}" != "post" ]; then
    echo "Invalid arg: ${stage}"
    exit ${ebadstage}
fi

# log something to stdout
info "running success_sample_pre_post.sh"

if [ "${stage}" = "pre" ]; then
    prehook
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during prehook"
    fi
fi

if [ "${stage}" = "post" ]; then
    posthook
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during posthook"
    fi
fi

exit ${rc}
```

故障示例

以下示例演示了如何处理挂机故障。

```
#!/bin/sh

# failure_sample_arg_exit_code.sh
#
# A simple failure hook script for testing purposes.
#
# args: [the exit code to return]
#
```

```
#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
error "script failed, returning exit code ${argexitcode}"

# exit with specified exit code
exit ${argexitcode}
```

详细故障示例

以下示例演示了如何处理挂机故障，并提供更详细的日志记录。

```
#!/bin/sh

# failure_sample_verbose.sh
#
# A simple failure hook script with args for testing purposes.
#
# args: [The number of lines to output to stdout]


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#
```

```
# log something to stdout
info "running failure_sample_verbose.sh"


# output arg value to stdout
linecount=$1
info "line count ${linecount}"

# write out a line to stdout based on line count arg
i=1
while [ "$i" -le ${linecount} ]; do
    info "This is line ${i} from failure_sample_verbose.sh"
    i=$(( i + 1 ))
done

error "exiting with error code 8"
exit 8
```

退出代码示例失败

以下示例显示了一个连接失败并显示退出代码。

```
#!/bin/sh

# failure_sample_arg_exit_code.sh
#
# A simple failure hook script for testing purposes.
#
# args: [the exit code to return]
#



#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}



#
# Writes the given information message to standard output
#
# $* - The message to write
```

```
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
error "script failed, returning exit code ${argexitcode}"

# exit with specified exit code
exit ${argexitcode}
```

失败后成功示例

以下示例显示了首次运行时发生故障的挂钩，但在第二次运行后仍会成功。

```
#!/bin/sh

# failure_then_success_sample.sh
#
# A hook script that fails on initial run but succeeds on second run for
testing purposes.
#
# Helpful for testing retry logic for post hooks.
#
# args: None
#
```

```
#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_success sample.sh"


if [ -e /tmp/hook-test.junk ] ; then
    info "File does exist.  Removing /tmp/hook-test.junk"
    rm /tmp/hook-test.junk
    info "Second run so returning exit code 0"
    exit 0
else
    info "File does not exist.  Creating /tmp/hook-test.junk"
    echo "test" > /tmp/hook-test.junk
    error "Failed first run, returning exit code 5"
```

```
    exit 5
fi
```

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.